

EFUSE 控制器设计方案

efuse_ctrl_top · DDIC 显示驱动芯片 · 片上 eFuse/OTP 存储控制器

文档版本 : V1.5

APB 时钟 : 100 MHz | efuse 时钟 : 24 MHz (12~36MHz 上电频偏)
存储容量 : 512 bit | 工作模式 : 烧写(1~8bit) / 读取 / 空闲

C:\Users\Pace_huang\Desktop\efuse_design

1 前言

1.1 编写目的

本文档描述应用于 DDIC（Display Driver IC，显示驱动芯片）的 eFuse 控制器 efuse_ctrl_top 的完整硬件设计方案。DDIC 芯片内部集成 MCU 或数字控制逻辑，通过 AMBA APB 总线与本控制器交互，实现对片上 eFuse（电子熔丝 / OTP 一次性可编程存储）的烧写、读取与空闲三种工作模式控制。

本文档按"前言 → 设计规范 → 启动场景 → 模块设计"组织，其中模块设计以顶层 efuse_ctrl_top 为入口，逐个子模块给出功能介绍、接口列表与时序图，作为 RTL 实现、验证、集成与交付的依据。

1.2 适用范围

本设计适用于需要片上非易失配置存储的 DDIC 芯片。eFuse 用于在晶圆级/封装级测试或出厂阶段一次性烧写芯片 ID、制造批次信息、面板/驱动校准参数（Trim）等数据；上电复位后由控制器自动加载这 512bit 数据，其中前 256bit 直接输出供系统即用，后 256bit 存入 APB 域内部寄存器（reg_file）供随机访问，并在加载完成后输出 load_done 加载完成信号。

eFuse 器件特性：上电后内部数据自动读出；只能从 0 烧写为 1（One-Time Programmable），不可擦除、不可写回 0。本控制器严格遵循上述物理特性设计控制逻辑。

1.3 术语与缩写

缩写	全称 / 含义
DDIC	Display Driver IC，显示驱动芯片
APB	Advanced Peripheral Bus，AMBA 高级外设总线
eFuse / OTP	电子熔丝 / One-Time Programmable 一次性可编程存储
CDC	Clock Domain Crossing，跨时钟域
FSM	Finite State Machine，有限状态机
内部寄存器 reg_file	上电加载后存放后 256bit 数据、供随机访问
Bank	eFuse 阵列按 64bit×8 组织的存储分组，一次读 8bit
sense amplifier	读出放大器，eFuse 阵列读取电路
charge pump	电荷泵，eFuse 烧写高压产生电路

RTL	Register Transfer Level，寄存器传输级
MTBF	Mean Time Between Failures，平均故障间隔时间（亚稳态）

1.4 参考资料

- ARM, AMBA 3 APB Protocol Specification (IHI 0024)
- eFuse/OTP 存储 IP 供应商数据手册（烧写/读取时序参数以实际 IP 为准）
- 芯片集成手册与 APB 地址分配表

2 设计规格

2.1 设计目标

- 通过 APB 从机接口接收主机配置，控制 eFuse 烧写（支持 1~8bit 一次写入）、读取（每次 8bit）与空闲三种工作模式
- 上电复位结束后自动加载 512bit：加载按"一次读 8bit × 64 次"完成，前 256bit 直接输出，后 256bit 存入内部寄存器（reg_file），全部加载完成后输出 load_done 加载完成信号
- 支持软件 RE_LOAD 重新加载全部 512bit（上电自动加载与软件重加载共用同一加载通路）
- 烧写与读取之间必须经过空闲状态过渡，禁止直接切换，保证 eFuse 阵列安全
- 通过同步握手（handshake）实现 100MHz APB 域与 24MHz efuse 域的跨时钟域可靠交互
- 提供 PSLVERR 错误指示，覆盖非法地址访问与无效寄存器读写
- 面积与功耗优先，适合 DDIC 内部集成

2.2 时钟与复位规格

项目	规格	说明
PCLK	100 MHz	APB 总线时钟，驱动 apb_slave / apb_fsm 等
efuse_clk	24 MHz	eFuse 器件独立时钟，允许 12~36MHz 上电频偏
PRESETn	异步复位	APB 域复位，低有效
efuse_rst_n	异步复位	eFuse 域复位，低有效，与 efuse_clk 同步释放
复位策略	异步复位、同步释放	每个时钟域各自例化 rst_sync，避免跨域亚稳态

两域频率不同（100MHz 与 24MHz，比例约 4.2:1）且相位无关，所有跨域信号必须经过 CDC 同步，详见第 4.5 节 cdc_sync。上电加载按"一次读 8bit × 64 次"进行，后 256bit 通过字节级双握手（load_req / load_data[7:0] / load_ack）从 24M 域跨到 100M 域写入内部寄存器 reg_file，见第 4.8 节。

2.3 存储规格与地址映射

EFUSE 器件可存储 512bit 数据，内部组织为 8 个 Bank（Bank0~Bank7），每个 Bank 64bit。读取时给出 6bit 基地址 i（0~63），一次并行返回 8bit：rdata[b] = Bank[b][i]，即一次访问读回 i, i+64, ..., i+7×64 共 8 个物理位。

位域	宽度	含义
基地址 i[5:0]	6bit	Bank 内偏移（列），范围 0~63
Bank 选择 b[2:0]	3bit	Bank 编号（= 位地址高 3 位），范围 0~7
物理位地址 addr[8:0]	9bit	addr = b×64 + i，范围 0~511

物理位地址(addr) 与基地址/Bank 的对应关系：读取给基地址 i，返回 8bit = {Bank7[i], ..., Bank0[i]}，即物理位 {i, i+64, i+2×64, ..., i+7×64}；烧写给完整 9bit 物理位地址 j，分解为基地址 i=j[5:0]、Bank 选择 b=j[8:6]。

基地址 i	展开的物理位地址 (=b×64+i)	返回数据 rdata[b]
i=0	0, 64, 128, 192, 256, 320, 384, 448	rdata[0]=Bank0[0], rdata[1]=Bank1[0], ... , rdata[7]=Bank7[0]
i=1	1, 65, 129, 193, 257, 321, 385, 449	rdata[0]=Bank0[1], ... , rdata[7]=Bank7[1]
i=63	63, 127, 191, 255, 319, 383, 447, 511	rdata[0]=Bank0[63], ... , rdata[7]=Bank7[63]

烧写操作支持每次烧写 1~8bit（软件通过 WDATA 寄存器 bit[7:0] 指定待烧写数据，CTRL[6:4] 指定本次烧写宽度 n，1≤n≤8），烧写地址为完整 9bit 物理位地址 prog_addr[8:0]。控制器在 efuse 域把本次烧写的第 j 位地址分解为 {bank=j[8:6], i=j[5:0]}，输出 efuse_addr=i、efuse_be=8'b1<<bank、efuse_wdata=data[j]；WDATA 中为 0 的位在对应周期不触发烧写。宽度 n>1 时，n 个物理位地址要求属于同一 Bank（即高 3 位相同），逐位烧写 data[0]~data[n-1]。

2.4 工作模式定义

模式	编码 (CTRL[1:0])	说明
空闲 Idle	2b00	不操作 eFuse，可接受模式切换与配置写入
烧写 Program	2b01	每次烧写 1~8bit（n 由 CTRL[6:4] 指定，n=prog_w+1），烧写完成前不接受其他烧写/读取
读取 Read	2b10	一次读取 8bit（访问基地址 i，返回 i, i+64, ..., i+7×64）
保留	2b11	保留，写入报 PSLVERR

关键约束：烧写模式与读取模式之间必须经过空闲状态过渡（APB 域经 IDLE_T、efuse 域经 IDLE），不能直接切换，保证 eFuse 阵列不会因模式混叠而损坏。

2.5 APB 接口规格

控制器作为 APB 从机接入芯片总线（APB4 两周期协议：SETUP 周期 PSEL=1/PENABLE=0，ACCESS 周期 PENABLE=1）：

信号	方向	宽度	说明
PCLK	input	1	APB 时钟，100MHz
PRESETn	input	1	APB 复位，低有效
PSEL	input	1	从机选择
PENABLE	input	1	APB 使能
PWRITE	input	1	写(1)/读(0)指示
PADDR	input	8	APB 地址，字节寻址
PWDATA	input	32	写数据
PRDATA	output	32	读数据
PREADY	output	1	准备好信号，跨域操作期间拉低插入等待
PSLVERR	output	1	传输错误指示（见 2.7 节）

2.6 eFuse 接口规格

控制器与 eFuse 器件本体之间的接口信号：

信号	方向	宽度	说明
efuse_clk	input	1	eFuse 时钟，24MHz（允许 12~36MHz 上电频偏）
efuse_rst_n	input	1	eFuse 域复位，低有效，异步复位同步释放
efuse_cs	output	1	片选（chip select）：高电平使能 eFuse 访问
efuse_rd	output	1	读使能，高有效

efuse_prog	output	1	烧写使能，高有效
efuse_addr[5:0]	output	6	基地址 i (0~63)：读与烧写时均为 Bank 内偏移
efuse_be[7:0]	output	8	烧写位选择 (byte enable)：烧写第 j 位时 = 8'b1<<j[8:6]，指示烧写哪个 Bank 的该位
efuse_wdata	output	1	烧写数据：烧写第 j 位时为 data[j]；该位=1 且 prog=1 时烧 1，=0 时不烧写
efuse_rdata[7:0]	input	8	读取返回的 8bit 数据
PSLVERR	output	1	APB 错误指示 (正常固定 0，异常时拉高)

注意：efuse_wdata 输出逻辑为 $efuse_wdata = efuse_prog \& efuse_cs \& data[j]$ （烧写第 j 位时若该位数据为 1 则选中即写 1，为 0 则本周期不触发烧写）；读取或空闲时 $efuse_wdata = 0$ 。烧写同时输出 $efuse_be = 8'b1 \ll bank$ 以选中对应 Bank 的该位。

2.7 寄存器定义与错误规则

寄存器基地址 BASE（由系统分配），以下偏移为相对 BASE 的地址，全部按 32bit 对齐：

偏移	名称	读写	位域	说明
0x00	CTRL	RW	[1:0]	工作模式：2b00=空闲，2b01=烧写，2b10=读取，2b11=保留
0x00	CTRL	RW	[2]	烧写启动 start_prog，写 1 触发烧写
0x00	CTRL	RW	[3]	读取启动 start_read，写 1 触发读取
0x00	CTRL	RW	[4]	RE_LOAD：写 1 触发一次重新加载全部 512bit，硬件自动清零
0x00	CTRL	RW	[6:4]	烧写宽度 n：n = prog_w[2:0] + 1，000~111 对应 1~8bit（一次烧写）
0x04	STATUS	RO	[0]	load_done：上电/RE_LOAD 512bit 加载完成标志（APB 域 reg_file 写完最后 1bit 本地置位，直接反映）
0x04	STATUS	RO	[1]	busy：控制器忙（加载/烧写/读取中）
0x04	STATUS	RO	[2]	mode[1:0] 回读（与 CTRL[1:0] 一致）
0x08	ADDR	RW	[8:0]	访问地址：烧写时完整 9bit 物理地址 prog_addr[8:0]，读取时低 6bit 为基地址 i (0~63)
0x0C	WDATA	RW	[7:0]	烧写数据 bit[7:0]（待烧写的 1~8bit 数据，取前 n 位用于本次烧写）
0x10	RDATA	RO	[7:0]	读取命令返回的 8bit 数据 rdata[7:0]
0x14	MEM_DATA0	RO	[31:0]	后 256bit 数据 bit[31:0]
0x18	MEM_DATA1	RO	[31:0]	后 256bit 数据 bit[63:32]
0x1C	MEM_DATA2	RO	[31:0]	后 256bit 数据 bit[95:64]
0x20	MEM_DATA3	RO	[31:0]	后 256bit 数据 bit[127:96]
0x24	MEM_DATA4	RO	[31:0]	后 256bit 数据 bit[159:128]
0x28	MEM_DATA5	RO	[31:0]	后 256bit 数据 bit[191:160]
0x2C	MEM_DATA6	RO	[31:0]	后 256bit 数据 bit[223:192]
0x30	MEM_DATA7	RO	[31:0]	后 256bit 数据 bit[255:224]
0x34~0x1FC	-	-	-	保留，访问报 PSLVERR

PSLVERR 错误检测规则（错误场景 E1~E5）：

编号	场景	说明
E1	访问保留地址	PADDR 落在 0x34~0x1FC 保留区间，或地址未对齐
E2	读只写/写只读	写 STATUS/RDATA/MEM_DATA*（只读）、读保留位等

		属性冲突
E3	非法 eFuse 地址	烧写物理地址 > 511，或读取基地址 i > 63
E4	模式冲突	烧写未完成时重复触发烧写，或读取未完成时切换模式
E5	模式切换错误	烧写↔读取未经过空闲直接切换（硬件强制经空闲，软件再次请求报错）

3 启动场景

3.1 DDIC 上电启动流程

DDIC 上电后，eFuse 控制器作为非易失配置源，负责在系统复位释放后自动把 eFuse 中预烧写的配置加载出来，供显示初始化使用。整体流程如下：

- 步骤 1：芯片上电，PCLK 与 efuse_clk 开始运行
- 步骤 2：PRESETn / efuse_rst_n 释放（各自域内经 rst_sync 同步释放）
- 步骤 3：efuse 域自动进入 LOAD 状态，按基地址 i=0~63 连续执行 64 次 8bit 读取（一次读 8bit），拼装出全部 512bit
- 步骤 4：前 256bit（Bank0~3）经 efuse_rdata[7:0] 直接输出；后 256bit（Bank4~7）在 efuse 域按 8bit 打包为 32 个字节，经字节级双握手（load_req / load_data[7:0] / load_ack）跨域写入 APB 域内部寄存器 reg_file
- 步骤 5：32 字节全部传输完成、APB 域写完最后 1bit 的同一时刻，在 APB 域直接置 load_done=1（本地反映到 STATUS[0]，无需再次跨域同步）
- 步骤 6：芯片主控轮询 STATUS[0]=1 后，即可通过 APB 直接随机访问 reg_file（读取/管理 eFuse 数据）

加载期间（LOAD 状态）控制器屏蔽一切 APB 触发请求，保证加载的原子性与完整性。

3.2 eFuse 加载过程

LOAD 状态共执行 64 次 8bit 读取（基地址 i=0~63），24MHz 下约 $64/24\text{MHz} \approx 2.7\mu\text{s}$ 。期间：

bit 范围	基地址	去向	说明
bit 0~255	i=0~63（Bank0~3）	直接输出	一次读 8bit = {Bank7[i],...,Bank0[i]}；前 256bit 经 efuse_rdata[7:0] 直接送到输出
bit 256~511	i=0~63（Bank4~7）	字节握手 → 100M reg_file	efuse 域按 8bit 打包为 32 个字节，经字节级双握手逐字节跨域，APB 域写入内部 reg_file（100MHz 域寄存器堆）
全部 512bit	i=0~63	—	64 次 8bit 读取拼装 512bit；APB 域写完最后 1bit 的同一时刻置 load_done=1 并直接输出

3.3 加载时序

复位释放后 efuse 域自动进入 LOAD。加载基地址计数器从 0 计到 63，efuse_cs=1、efuse_rd=1、地址按基地址 i 递增，每次读取返回 8bit（i, i+64, ..., i+7×64）；前 256bit 直出（efuse_rdata 直接输出）；后 256bit 在 efuse 域每累计 8bit 打包为 1 字节，通过字节级双握手逐字节送到 APB 域，APB 域写状态机将 32 字节写入内部寄存器 reg_file；写完最后 1bit（第 32 字节）的同一时刻，APB 域置 load_done=1 并直接反映到 STATUS[0]。加载时序见图 1（双时钟域）。

上电加载时序 (双域: 24M 一次读8bit×64次 → 字节握手 → 100M reg_file + load_done) • EDA 仿真波形

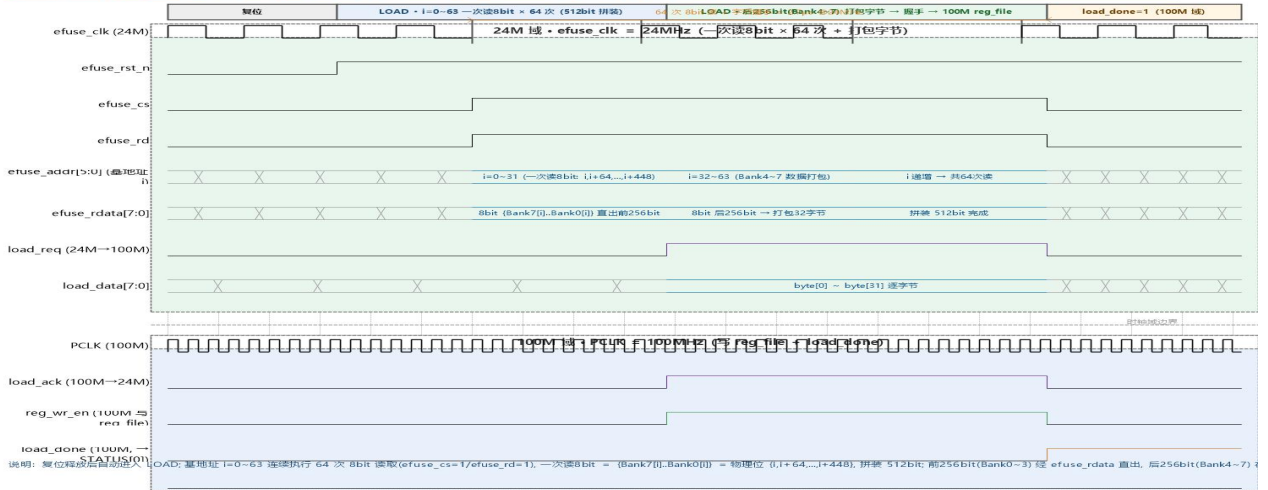


图 1 上电加载时序 (双域: 24M 读出 → 字节握手 → 100M reg_file + load_done)

3.4 启动完成后工作场景

load_done 置位后，系统进入正常运行，可通过 APB 接口执行：

- 读取场景：主控写 ADDR（基地址 i）→ 写 CTRL[3]=start_read → 控制器在 24M 域执行一次 8bit 读（i, i+64, ..., i+7×64）→ 主控读 RDATA 寄存器取得 8bit
- 烧写场景（仅出厂/测试阶段）：主控写 ADDR（完整 9bit 物理地址 j={bank,i}）与 WDATA（bit[7:0]=待烧数据）→ 写 CTRL（含烧写宽度 n 与 start_prog=1）→ 控制器在 efuse 域把物理地址分解为 {bank=j[8:6], i=j[5:0]}，按 n 个物理位逐位输出 efuse_cs=1、efuse_prog=1、efuse_addr=i、efuse_be=8b1<<bank、efuse_wdata=data[j]，仅 data[j]=1 的位触发烧写
- 模式切换：烧写与读取之间必须显式写入 CTRL[1:0]=2b00（空闲）过渡，硬件 FSM 亦强制此约束

4 模块设计

4.1 顶层模块 efuse_ctrl_top

功能：efuse_ctrl_top 为整个 eFuse 控制器的最顶层，对外呈现 APB 从机接口与 eFuse 器件接口，内部划分为两个时钟域、多个功能子模块，负责总体例化、互联与顶层信号分配。

接口列表：

信号	方向	宽度	说明
PCLK	input	1	APB 时钟 100MHz
PRESETn	input	1	APB 复位，低有效
PSEL	input	1	APB 从机选择
PENABLE	input	1	APB 使能
PWRITE	input	1	APB 读写指示
PADDR[7:0]	input	8	APB 地址
PWDATA[31:0]	input	32	APB 写数据
PRDATA[31:0]	output	32	APB 读数据
PREADY	output	1	APB 准备好
PSLVERR	output	1	APB 错误
efuse_clk	input	1	eFuse 时钟 24MHz（12~36MHz 频偏）
efuse_rst_n	input	1	eFuse 域复位，低有效
efuse_cs	output	1	片选（读/烧写有效）
efuse_rd	output	1	读使能

efuse_prog	output	1	烧写使能
efuse_addr[5:0]	output	6	基地址 i（读：0~63；烧写：物理地址低 6bit j[5:0]）
efuse_be[7:0]	output	8	烧写位选择（=8b1<<bank，仅烧写有效）
efuse_wdata	output	1	烧写数据（第 j 位 = data[j]，=1 且 be 选中时该周期烧 1）
efuse_rdata[7:0]	input	8	eFuse 读数据
load_done	output	1	上电 512bit 加载完成信号（后 256bit 写入内部寄存器完成后输出）

顶层框图：

EFUSE 控制器顶层架构图 (efuse ctrl top)

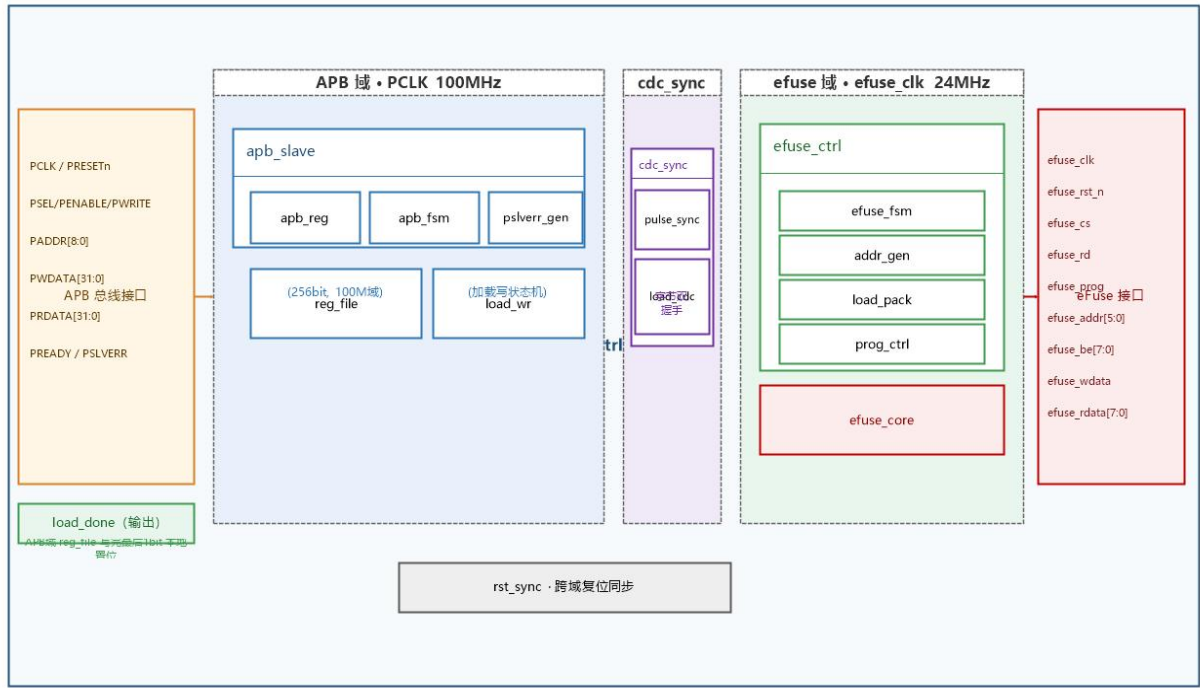


图 2 顶层架构框图 (efuse_ctrl_top)

子模块例化关系：

子模块	所属域	功能说明
apb_slave	PCLK 100MHz	APB 从机接口：协议解析、寄存器组、地址译码、PREADY/PSLVERR 产生
└─ apb_reg	PCLK 100MHz	寄存器组与译码：CTRL/STATUS/ADDR/WDATA/RDATA/MEM_DATA* 等寄存器读写逻辑（含烧写宽度 n）
└─ apb_fsm	PCLK 100MHz	模式控制状态机：空闲/烧写(1~8bit)/读取三模式，强制烧写↔读取经空闲过渡
└─ pslverr_gen	PCLK 100MHz	错误检测：按 2.7 节 E1~E5 规则生成 PSLVERR
reg_file	PCLK 100MHz	内部寄存器堆：256bit（32×8），存放后 256bit，供 APB 同域随机访问
└─ load_wr	PCLK 100MHz	加载写状态机：接收字节握手，把 32 字节写入 reg_file，写完最后 1bit 置 load_done
cdc_sync	跨时钟域	脉冲/电平/字节握手同步器：请求、应答、状态、加载数据在 100M 与 24M 域间可靠传递

└─ pulse_sync(req)	100M→24M	将 APB 域读取/烧写请求单脉冲同步到 efuse 域
└─ pulse_sync(ack)	24M→100M	将 efuse 域操作完成应答同步回 APB 域
└─ load_cdc	24M→100M	字节级双握手：load_req/load_data[7:0]/load_ack，传输加载后 256bit（32 字节）
└─ level_sync	双向	模式/忙等电平信号跨域同步
efuse_ctrl	efuse_clk 24MHz	eFuse 操作控制器：加载(8bit×64 拼装)/读取/烧写(1~8bit)流程控制与地址生成
└─ efuse_fsm	efuse_clk 24MHz	操作状态机：LOAD/IDLE/READ/PROG(按宽度 n)，读/写经 IDLE 过渡
└─ addr_gen	efuse_clk 24MHz	地址生成：加载 i=0~63、读取基地址 i、烧写 j={bank,i} 分解
└─ prog_ctrl	efuse_clk 24MHz	烧写控制：按宽度 n 逐位输出 efuse_cs/prog/be/wdata=data[j]
└─ load_pack	efuse_clk 24MHz	加载打包：前 256bit 直出，后 256bit 按 8bit 打包成字节并经 load_cdc 发出
rst_sync	全局	异步复位同步释放器，为各域提供干净复位
efuse_core	efuse_clk 24MHz	eFuse 存储阵列本体（sense amplifier + 编程电荷泵），受控于 efuse_ctrl

4.2 子模块 apb_slave（APB 从机接口）

功能：实现标准 APB4 从机协议，完成寄存器读写、地址译码、忙等待（PREADY）与错误上报（PSLVERR），是软件与控制器之间的唯一接口。

接口列表：

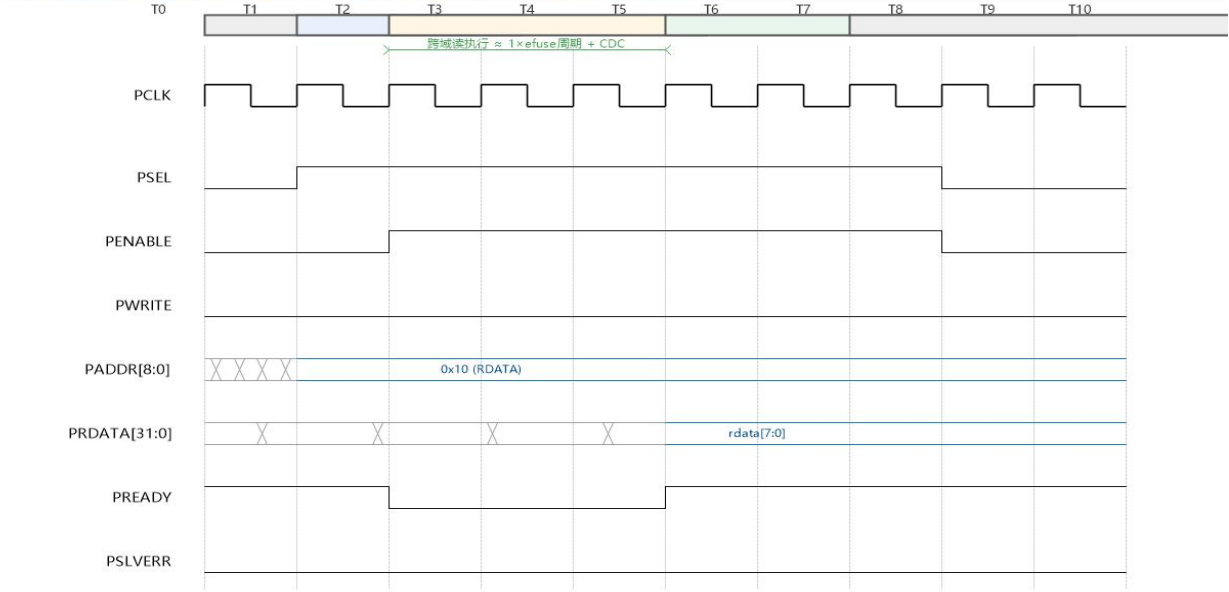
信号	方向	宽度	说明
PCLK / PRESETn	input	1 / 1	时钟 / 复位
PSEL / PENABLE / PWRITE	input	1 / 1 / 1	APB 控制信号
PADDR[7:0]	input	8	地址（与寄存器偏移对应）
PWDATA[31:0]	input	32	写数据
PRDATA[31:0]	output	32	读数据
PREADY	output	1	忙时拉低等待
PSLVERR	output	1	错误指示
cfg_mode[1:0]	output	2	当前工作模式
req_read / req_prog	output	1 / 1	读取 / 烧写启动请求（单脉冲）
apb_addr[8:0]	output	9	本次访问地址锁存（读：低 6bit 为基地址 i；烧写：完整 9bit 物理地址 j）
prog_w[2:0]	output	3	烧写宽度 n（1~8）
prog_data[7:0]	output	8	待烧写数据 data[7:0]
stat_load_done / stat_busy	input	1 / 1	来自 APB 域状态（load_done 本地产生，busy 经同步）

行为要点：

- 采用 APB4 标准：PSEL & PENABLE & ~PWRITE 为读，PSEL & PENABLE & PWRITE 为写
- 写 CTRL[1:0] 后立即开始模式仲裁，若目标模式与当前不同则进入过渡流程（经 IDLE_T）
- 写 CTRL 时锁存烧写宽度 n（CTRL[6:4]）与 WDATA[7:0]（待烧写数据）
- 控制器 busy 且访问 WDATA/ADDR 时 PREADY 拉低等待操作完成
- 后 256bit reg_file 位于本模块同域（PCLK），加载由 load_wr 状态机写入，APB 读 reg_file 为同域同步读，零等待、无跨域
- 所有错误场景（2.7 节）在本模块统一产生 PSLVERR

时序图：

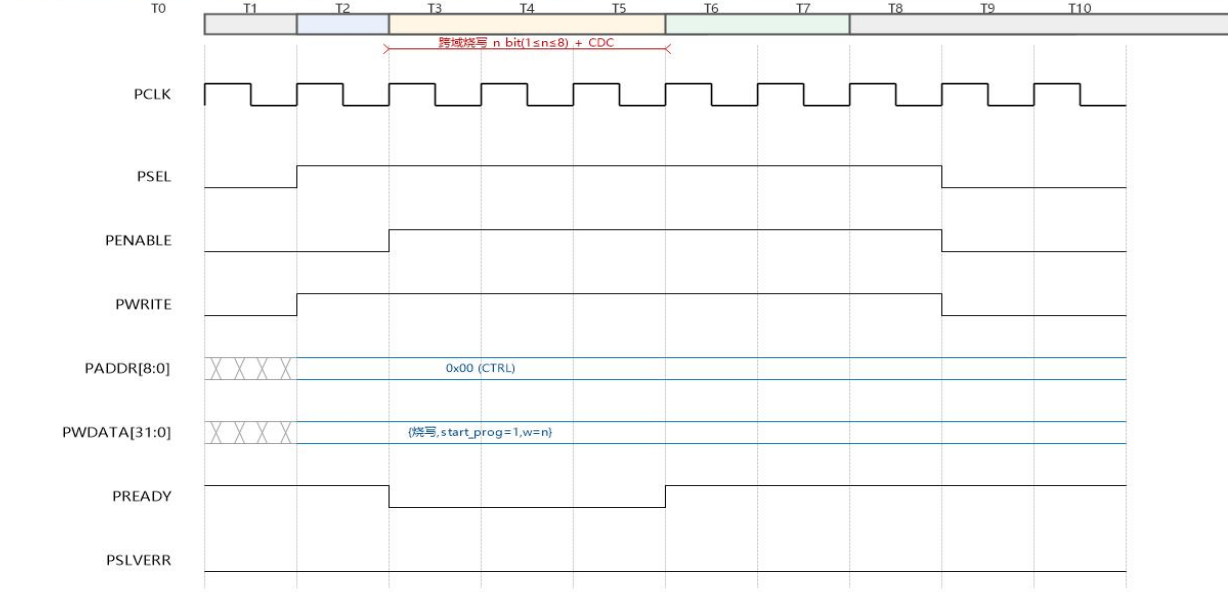
APB 读时序 (读取8bit, 含等待周期 PREADY=0) • EDA 仿真波形



说明: T1=SETUP(PSEL=1); T2~T4=ACCESS 等待 PREADY=0(efuse 在 24M 域执行一次 8bit 读取+ CDC 同步); T5=读取完成 PREADY=1, PRDATA 有效并保持到 T6

图 3 APB 读时序 (读取 8bit, 含等待周期 PREADY=0)

APB 写时序 (烧写 1~8bit, 含等待周期 PREADY=0) • EDA 仿真波形



说明: 写 CTRL 触发烧写, PWDATA 含模式=烧写、start_prog=1、烧写宽度 n(1~8); 控制器在 efuse 域逐位烧写 WDATA[0]~WDATA[n-1], 期间 PREADY=0 等待

图 4 APB 写时序 (烧写 1~8bit, 含等待周期 PREADY=0)

4.3 子模块 apb_fsm (模式控制状态机)

功能：运行在 PCLK=100MHz 域，管理三种工作模式并强制"烧写↔读取必须经过空闲状态 (IDLE_T) 过渡"，从设计上杜绝直接切换。

接口列表：

信号	方向	宽度	说明
----	----	----	----

PCLK / PRESETn	input	1 / 1	时钟 / 复位
cfg_wr	input	1	写 CTRL 指示（来自 apb_reg）
cfg_mode_w[1:0]	input	2	软件写入的目标模式
prog_busy / read_busy	input	1 / 1	efuse 域操作忙指示
ack_prog / ack_read	input	1 / 1	efuse 域操作完成应答（同步后）
prog_w[2:0]	input	3	本次烧写宽度 n（1~8）
state_out[3:0]	output	4	当前模式状态（IDLE/PROG/READ/IDLE_T）
mode[1:0]	output	2	输出给 CDC 与 pslverr_gen 的当前模式

FSM 状态转换图：

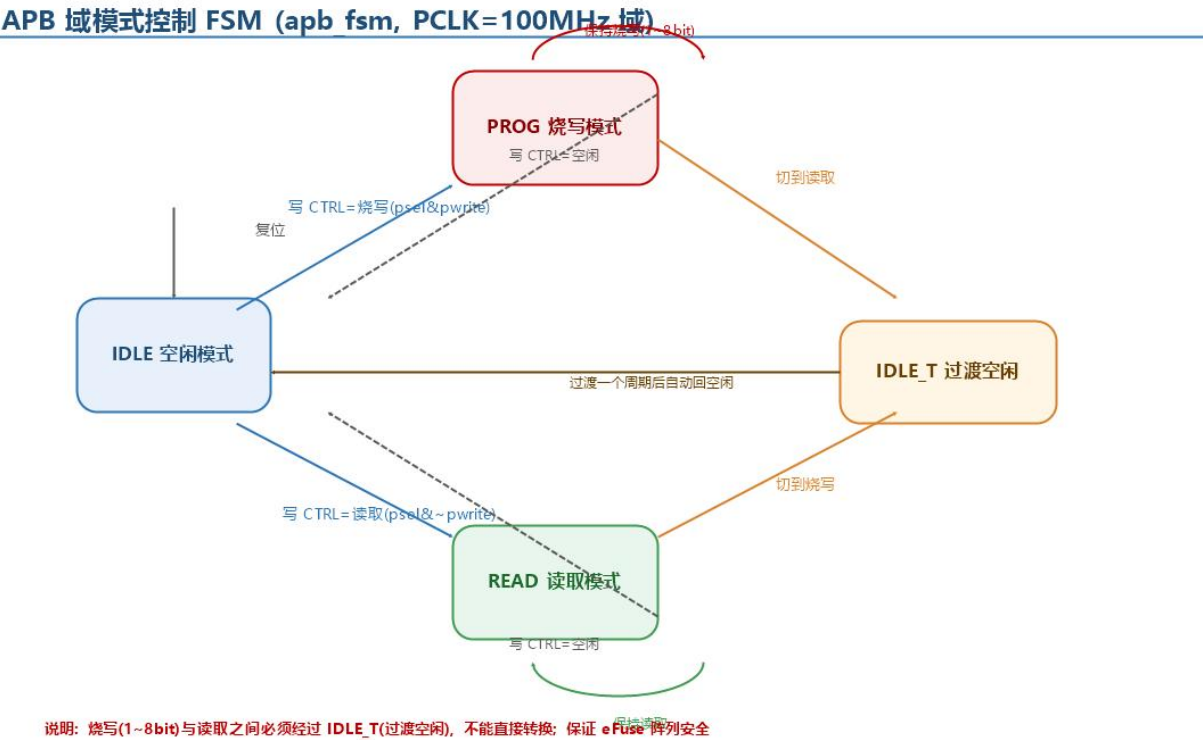


图 5 APB 域模式控制 FSM (apb_fsm)

状态	含义	进入条件	离开条件
IDLE	空闲模式，可接受任意请求	复位 / 过渡完成	写 CTRL 选烧写或读取
PROG	烧写模式	IDLE 下写 CTRL=烧写	切读取（需经 IDLE_T） / 写 CTRL=空闲
READ	读取模式	IDLE 下写 CTRL=读取	切烧写（需经 IDLE_T） / 写 CTRL=空闲
IDLE_T	过渡空闲	PROG→READ 或 READ→PROG 切换时	停留一个 PCLK 周期后自动回 IDLE

关键规则：PROG 与 READ 之间不能直接跳转，必须先经 IDLE_T（过渡空闲）中转，保证 eFuse 阵列安全。

4.4 子模块 pslverr_gen（错误检测）

功能：根据 APB 地址、模式与操作类型判断访问是否非法，输出 PSLVERR。

接口列表：

信号	方向	宽度	说明
PADDR[7:0]	input	8	APB 地址

PSEL / PENABLE / PWRITE	input	1 / 1 / 1	APB 控制信号
cfg_mode[1:0]	input	2	当前模式
prog_busy / read_busy	input	1 / 1	操作忙指示
PSLVERR	output	1	错误指示（组合逻辑，ACCESS 周期生效）

- 实现：纯组合译码，按 2.7 节 E1~E5 规则产生，无状态寄存器，面积开销极小
- PSLVERR 拉高一个 PCLK 周期；PREADY 仍正常拉高以完成传输，总线不挂死

4.5 子模块 cdc_sync（跨时钟域同步）

功能：在 100MHz APB 域与 24MHz efuse 域之间传递请求脉冲、完成应答、电平状态与加载数据，保证所有跨域信号无亚稳态、无丢请求/丢应答。

接口列表（除下述加载字节握手外，均与 V1.2 一致）：

信号	方向	宽度	说明
req_read	input	1	100M 域读取请求（单脉冲）
req_prog	input	1	100M 域烧写请求（单脉冲）
rd_req_24m	output	1	同步到 24M 域的读取请求脉冲
prog_req_24m	output	1	同步到 24M 域的烧写请求脉冲
ack_read_24m	input	1	24M 域读取完成应答
ack_prog_24m	input	1	24M 域烧写完成应答
ack_read	output	1	同步回 100M 域的读取完成脉冲
ack_prog	output	1	同步回 100M 域的烧写完成脉冲
load_req	input	1	24M 域加载字节请求（load_cdc 源侧，电平）
load_data[7:0]	input	8	24M 域加载字节数据（load_cdc 源侧）
load_ack	output	1	100M 域加载字节应答（load_cdc 目的侧，电平）
load_req_100m	output	1	同步到 100M 域的加载字节请求
load_data_100m[7:0]	output	8	同步到 100M 域的加载字节数据
load_ack_24m	input	1	24M 域加载字节应答（经同步回送）
busy_24m	input	1	24M 域忙电平
busy	output	1	同步到 100M 域的忙电平

同步器结构：

路径	源域→目的域	同步器类型	传递内容
req_read	100M→24M	pulse_sync	读取请求
req_prog	100M→24M	pulse_sync	烧写请求
ack_read / ack_prog	24M→100M	pulse_sync	操作完成应答
load_req / load_data	24M→100M	load_cdc（字节级双握手）	加载后 256bit（32 字节）
load_ack	100M→24M	load_cdc（字节级双握手）	加载字节应答
busy / mode	24M→100M	level_sync（2 级 FF）	忙状态 / 当前模式

字节级双握手（load_cdc）时序：24M 源侧置 load_req=1 并给出 load_data[7:0] → 100M 侧两级同步采样 → 置 load_ack=1 → 源侧撤 load_req → 目的侧撤 load_ack，完成 1 字节传输；重复 32 次传完 32 字节。load_done 由 100M 域 reg_file 写状态机（load_wr）在写完最后 1bit 时本地置位，直接反映到 STATUS[0]，无需跨域同步。

通用 4 相握手时序图（脉冲同步器，req/ack 路径）：

跨时钟域 (CDC) 握手时序: 100MHz APB 域 → 24MHz efuse 域 • EDA 仿真波形

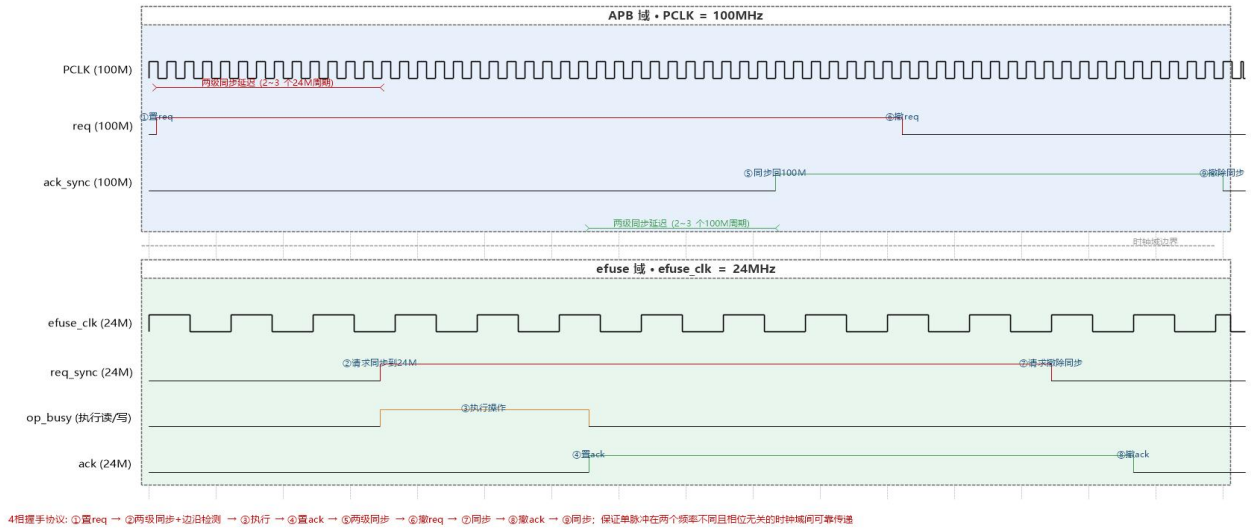


图 6 CDC 握手时序 (100M 域 ↔ 24M 域, req/ack 脉冲同步)

握手关键点:

- 请求方 (100M 域) 置 req=1 → 接收方 (24M 域) 两级触发器同步 + 边沿检测生成请求脉冲 → 执行操作
- 操作完成置 ack=1 → 100M 域两级同步 + 边沿检测生成完成脉冲 → 请求方撤 req=0
- 接收方检测到 req 撤除后拉低 ack, 完成一次完整握手
- 加载数据通路独立使用字节级双握手 (load_cdc), 与 req/ack 脉冲握手互不干扰
- 由于频率比 $100M/24M \approx 4.2$, 采样裕度充足; 两级同步器均满足 MTBF 要求

4.6 子模块 efuse_ctrl (含 efuse_fsm, eFuse 操作控制器)

功能: 运行在 efuse_clk=24MHz 域, 是 eFuse 域的核心, 负责上电加载 (LOAD) 与读取/烧写操作的执行, 并产生全部 eFuse 接口时序。

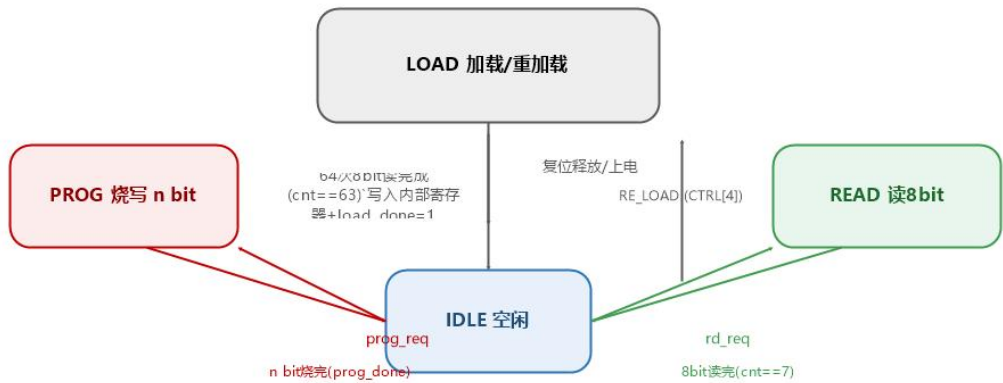
接口列表:

信号	方向	宽度	说明
efuse_clk / efuse_rst_n	input	1 / 1	eFuse 时钟 / 复位
rd_req_24m	input	1	同步后的读取请求脉冲
prog_req_24m	input	1	同步后的烧写请求脉冲
apb_addr[8:0]	input	9	APB 传入完整物理地址 j (读取时低 6bit 为基地址 i; 烧写时高 3bit 为 bank)
prog_data[7:0]	input	8	烧写数据 (bit[7:0], 取前 n 位用于本次烧写)
prog_w[2:0]	input	3	烧写宽度 n (1~8)
efuse_cs	output	1	片选 (读/烧写有效)
efuse_rd	output	1	读使能
efuse_prog	output	1	烧写使能
efuse_addr[5:0]	output	6	基地址 (读: i; 烧写: j[5:0])
efuse_be[7:0]	output	8	烧写位选择 (=8b1<<bank)
efuse_wdata	output	1	烧写数据 (第 j 位 = prog_data[j])
efuse_rdata[7:0]	input	8	eFuse 读数据
rdata[7:0]	output	8	打包后的 8bit 读数据 (回传 APB 域)
ack_read_24m / ack_prog_24m	output	1 / 1	操作完成应答
busy_24m	output	1	忙电平
load_req	output	1	加载字节请求 (load_cdc 源侧, 后 256bit 打包发

			出)
load_data[7:0]	output	8	加载字节数据
load_ack_24m	input	1	加载字节应答（经同步回送）

内部 efuse_fsm 状态转换图：

eFuse 域操作 FSM (efuse fsm, efuse clk=24MHz 域)



说明：READ 与 PROG 之间必须经过 IDLE(空闲) 过渡，不能直接转换；LOAD 阶段屏蔽一切 APB 请求；写 CTRL[4]=RE_LOAD 可再次进入 LOAD 重加载(与上电共用通路)

图 7 eFuse 域操作 FSM (efuse_fsm)

状态	含义	进入条件	离开条件
LOAD	上电加载	复位释放（自动） / RE_LOAD=1	64 次 8bit 读取完成（cnt==63）回 IDLE
IDLE	空闲	加载完成 / 操作完成	收到 rd_req / prog_req / RE_LOAD
READ	读取 8bit	IDLE 下 rd_req	一次 8bit 读完回 IDLE
PROG	烧写 n bit(1~8)	IDLE 下 prog_req	n bit 烧完（prog_done, cnt==n-1）回 IDLE

关键规则：READ 与 PROG 之间必须经过 IDLE（空闲）过渡，与 APB 域 IDLE_T 规则对应；LOAD 阶段屏蔽一切 APB 请求。PROG 阶段宽度 n 由 prog_w[2:0] 决定，逐位烧写 data[0]~data[n-1]，cnt 计满 n-1 即 prog_done。efuse_fsm 建议采用独热编码（LOAD=4b0001, IDLE=4b0010, READ=4b0100, PROG=4b1000），便于时序分析。

4.7 子模块 addr_gen（地址生成）

功能：生成 eFuse 操作所需地址，覆盖加载、读取与烧写三种场景。

接口列表：

信号	方向	宽度	说明
clk / rst_n	input	1 / 1	eFuse 域时钟 / 复位
fsm_state[3:0]	input	4	来自 efuse_fsm 的当前状态
apb_addr[8:0]	input	9	APB 传入完整物理地址 j（j={bank,i}）
prog_w[2:0]	input	3	烧写宽度 n（1~8）
efuse_addr[5:0]	output	6	输出的 eFuse 基地址
efuse_be[7:0]	output	8	烧写位选择（=8b1<<bank）

load_cnt[5:0]	output	6	加载基地址计数器（0~63）
addr_err	output	1	地址非法指示（反馈 pslverr_gen）

- 加载阶段：内部 6bit 基地址计数器 cnt_load[5:0] 从 0 递增到 63，输出 efuse_addr = cnt_load[5:0]（一次读 8bit）
- 读取阶段：输出 efuse_addr = i（APB 传入基地址），一次读出 8bit（i, i+64, ..., i+7×64）
- 烧写阶段：锁存 APB 传入的物理地址 j={bank, i}, efuse_addr = j[5:0], efuse_be = 8b1<<bank; 按宽度 n 依次取 j[0]~j[n-1]（要求 n 个物理位位于同一 Bank 内）
- 地址合法性检查：物理地址 > 511 或基地址 i > 63（越界）时产生 addr_err，反馈到 pslverr_gen

4.8 子模块 load_pack / load_cdc / load_wr + reg_file（上电加载与内部寄存器）

功能：实现"前 256bit 直接输出，后 256bit 存入内部寄存器（reg_file）"的上电加载要求。reg_file 位于 100MHz APB 域，与 APB 寄存器同域，加载数据经字节级双握手（load_cdc）从 24M 域跨到 100M 域，由 load_wr 状态机写入 reg_file，并在写完最后 1bit 的同一时刻由 APB 域本地产生 load_done 加载完成信号（直接反映到 STATUS[0]）。

接口列表（load_pack，24M 域）：

信号	方向	宽度	说明
clk / rst_n	input	1 / 1	eFuse 域时钟 / 复位
fsm_state[3:0]	input	4	当前状态（LOAD 时使能）
efuse_rdata[7:0]	input	8	eFuse 读数据
load_cnt[5:0]	input	6	加载基地址计数器（0~63）
load_req	output	1	字节请求（load_cdc 源侧）
load_data[7:0]	output	8	打包后的 8bit 字节数据
load_ack_24m	input	1	字节应答（经 load_cdc 回送）

接口列表（load_cdc，跨域桥）：

信号	方向	宽度	说明
clk_24m / clk_100m	input	1 / 1	双域时钟
rst_24m_n / rst_100m_n	input	1 / 1	双域复位
src_req / src_data[7:0]	input	1 / 8	24M 源侧请求 / 字节数据
dst_req / dst_data[7:0]	output	1 / 8	100M 目的侧请求 / 字节数据
dst_ack	output	1	100M 目的侧应答（回送源侧）
src_ack	input	1	24M 源侧应答（经同步回送）

接口列表（load_wr，100M 域）：

信号	方向	宽度	说明
clk / rst_n	input	1 / 1	APB 域时钟 / 复位
dst_req / dst_data[7:0]	input	1 / 8	同步后的加载字节请求 / 数据
dst_ack	output	1	字节应答
reg_wr_en	output	1	reg_file 写使能（32 字节写入期间有效）
reg_waddr[7:0]	output	8	reg_file 写地址（0~255，每次字节写 8bit）
reg_wdata[7:0]	output	8	写入字节数据
load_done	output	1	加载完成（写完最后 1bit 同一时刻置位，直接反映 STATUS[0]）

接口列表（reg_file，100M 域）：

信号	方向	宽度	说明
----	----	----	----

clk	input	1	APB 域时钟 100MHz
wr_en	input	1	写使能（仅加载阶段 load_wr）
waddr[7:0]	input	8	写地址 0~255
wdata[7:0]	input	8	写字节数据
raddr[7:0]	input	8	读地址（APB 随机访问）
rdata[7:0]	output	8	读出的 8bit 数据（同域同步读，零等待）

- reg_file 实现：256bit 寄存器堆（32 个 8bit 字），加载时按字节（8bit）写入，APB 访问时按地址随机读，与 APB 寄存器同属 100MHz 域
- 加载流程：24M 域 LOAD 按基地址 $i=0\sim63$ 执行 64 次 8bit 读取拼装 512bit；后 256bit（Bank4~7）load_pack 每累计 8bit 打包为 1 字节 → load_cdc 字节级双握手逐字节发送 → load_wr 收到字节写入 reg_file，共 32 字节
- 字节级双握手：源侧置 load_req=1 + load_data → 目的侧两级同步采样并置 load_ack=1 → 源侧撤 load_req → 目的侧撤 load_ack，完成 1 字节；重复 32 次
- load_done 在 load_wr 写完第 32 字节（最后 1bit）的同一时刻置位并直接反映 STATUS[0]，无需跨域同步；加载时序见图 1（第 3.3 节，双时钟域）

4.9 子模块 rst_sync（复位同步）

功能：对全局异步复位做同步释放，为 PCLK 域与 efuse_clk 域分别提供干净的复位信号。

接口列表：

信号	方向	宽度	说明
clk	input	1	本域时钟
rst_n_async	input	1	全局异步复位（低有效）
rst_n_sync	output	1	同步释放后的本域复位（低有效）

- 两级触发器结构：rst_n 异步置位、同步释放
- 每个时钟域各自例化一个 rst_sync，避免跨域复位亚稳态
- eFuse 域复位释放后自动进入 LOAD 状态，开始上电加载

4.10 子模块 efuse_core（存储阵列 IP）

功能：eFuse 存储阵列本体，包含读出放大器（sense amplifier）与编程电荷泵（charge pump），由 efuse_ctrl 的接口时序驱动。本设计将其作为受控 IP 处理，烧写/读取时序参数以 IP 供应商数据手册为准。

接口列表：

信号	方向	宽度	说明
efuse_clk	input	1	eFuse 时钟 24MHz
efuse_cs	input	1	片选，高有效
efuse_rd	input	1	读使能，高有效
efuse_prog	input	1	烧写使能，高有效
efuse_addr[5:0]	input	6	基地址（读/烧写均为低 6bit 基地址 i）
efuse_be[7:0]	input	8	烧写位选择（ $=8b1\ll bank$ ，仅烧写有效）
efuse_wdata	input	1	烧写数据（第 j 位 = data[j]，=1 且 be 选中时烧 1）
efuse_rdata[7:0]	output	8	读取返回的 8bit 数据

eFuse 接口操作时序图：

eFuse 接口时序 (24M 域): 一次读8bit + 空闲过渡 + n bit烧写(1≤n≤8) • EDA 仿真波形

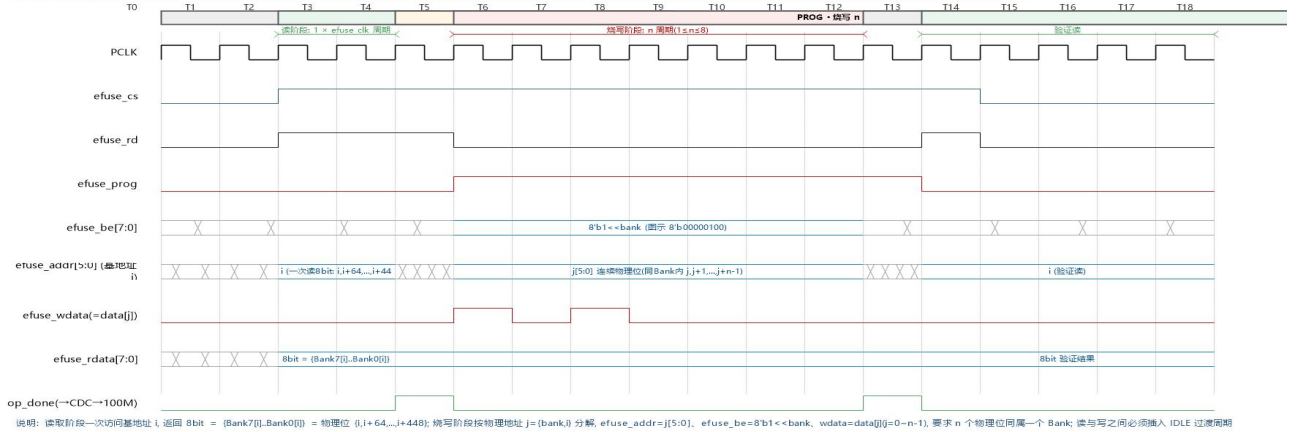


图 8 eFuse 接口时序 (读 8bit + 空闲过渡 + 烧写 n bit, 1≤n≤8)

要点: 读取阶段一次访问基地址 i , 返回 8bit ($i, i+64, \dots, i+7 \times 64$); 读与写之间必须插入空闲周期; 烧写阶段按物理地址 $j=\{\text{bank}, i\}$ 逐位输出, $\text{efuse_cs}=1$ 、 $\text{efuse_be}=8b1 \ll \text{bank}$ 、 $\text{efuse_addr}=i$ 、 $\text{wdata}=\text{data}[j]$, 仅 $\text{data}[j]=1$ 且 be 选中的位触发烧写 (电荷泵升压+烧断)。

5 附录

5.1 寄存器访问速查

偏移	名称	读/写	模式依赖
0x00	CTRL	RW	空闲/烧写/读取均可写模式
0x04	STATUS	RO	任何模式可读
0x08	ADDR	RW	锁存基地址 i (读/写物理地址低 6bit)
0x0C	WDATA	RW	烧写模式 bit[7:0] 为待烧写的 1~8bit 数据
0x10	RDATA	RO	读取模式返回一次读出的 8bit 数据
0x14~0x30	MEM_DATA0~7	RO	后 256bit 数据 (每次读 32bit, 对应 MEM_DATA0~7)

5.2 设计权衡

- 采用"APB 域模式 FSM + eFuse 域操作 FSM"双 FSM 结构, 职责分离清晰、便于验证; 代价是增加一组 CDC 握手逻辑
- 存储采用 8 Bank $\times$ 64bit 组织: 一次读 8bit ($i, i+64, \dots, i+7 \times 64$), 上电加载 64 次即拼装 512bit, 较逐位串行读出显著缩短加载时间 (约 64 个 eFuse 周期)
- 烧写支持 1~8bit: 软件一次写入宽度可配, 物理地址 $j=\{\text{bank}, i\}$, $\text{efuse_be}=8b1 \ll \text{bank}$ 选中 Bank, 控制器按宽度 n 逐位烧写 (要求 n 个物理位位于同一 Bank), 兼顾灵活性与阵列安全 (每周期仅烧 1bit)
- 前 256bit 直出 + 后 256bit 内部寄存器 (reg_file 位于 100MHz APB 域): 满足上电即用前 256bit 的低延迟要求, 后 256bit 与 APB 寄存器同域、APB 随机访问零等待无跨域; 加载数据经字节级双握手 (load_cdc) 跨域, load_done 由 APB 域本地产生
- PREADY 插入等待而非拒绝访问: 保证操作原子性, 简化软件驱动

5.3 文档修订记录

版本	日期	说明
V1.0	2026-08-09	初始版本: 完整方案设计
V1.1	2026-08-09	按 DDIC 场景重构: 前言/设计规格/启动场景/模块设计, 顶层 efuse_ctrl_top + 各子模块 (功能介绍/接口列表/时序图), 新增 CDC 握手时序图

V1.2	2026-08-09	框架图与时序图精细化：框架图模块名与第 4 章完全一致并划分 APB/efuse 时钟域；时序图增加阶段条、周期对齐、时间刻度与参数标注
V1.3	2026-08-09	烧写升级为 1~8bit（CTRL[6:4] 宽度 n + DATA[7:0] 数据，efuse_addr 扩展 9bit 连续烧写）；上电加载后 256bit 改存内部寄存器 reg_file，写完最后 1bit 同时输出 load_done；全部时序图按统一规范重画
V1.4	2026-08-09	架构优化：后 256bit reg_file 由 24M 域移至 100MHz APB 域（与 APB 寄存器同域，APB 随机访问零等待无跨域）；加载跨域改为字节级双握手 load_cdc（load_pack→load_cdc→load_wr），load_done 由 APB 域本地产生直接反映 STATUS[0]
V1.5	2026-08-09	参考 DeepSeek 方案升级：存储改为 8 Bank×64bit 组织、一次读 8bit，上电加载改 8bit×64 次拼装 512bit；新增 RE_LOAD（CTRL[4]）重加载、RDATA（0x10）读寄存器、MEM_DATA0~7（0x14~0x30）后 256bit 只读寄存器；eFuse 接口改为 efuse_cs/efuse_be[7:0]/efuse_addr[5:0]（基地址），烧写物理地址 j={bank,i} 分解；保留 1~8bit 烧写宽度与字节级双握手架构